Universität Konstanz

Introduction to Computing for the Social Sciences

Session 11 – Databases

David Garcia

Winter Semester 2026-2027

Course Outline

- Session 1: Introduction
- Session 2: Information Coding
- Session 3: Data Structures
- Session 4: Software
- Session 5: Algorithms
- Session 6: Recursion
- Session 7: Sorting Algorithms
- Session 8: Time Complexity
- Session 9: Theoretical Computer Science
- Session 10: Machine Learning
- Session 11: Databases
  - Tutorial: Databases (swap)
  - Exercise: DB + projects Q&A
- Session 12: Parallel Programming
- Session 13: Social Science Applications
- Session 14: Exam Review

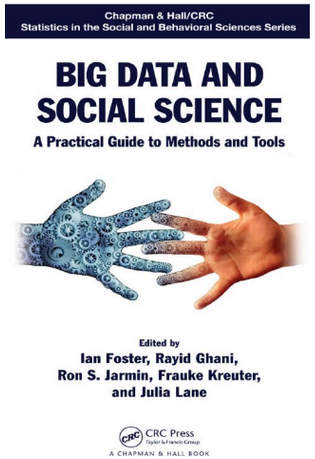
Overview

- Databases
- Relational Databases
- Structured Query Language (SQL)

Overview

- Databases
- Relational Databases
- Structured Query Language (SQL)
- NoSQL databases

Recommended: Chapter 4



The 15 841 cases that fell off the end of a spreadsheet

Universität Konstanz

MOTIVATING EXAMPLE

Between 25 September and 2 October 2020, Public Health England lost **15 841 positive COVID-19 test results**. Labs sent results as CSV; the collating process used an old spreadsheet format whose worksheets hold **65 536 rows**. When a file exceeded that, the extra rows were simply not read. **No error appeared.**

15 841
positive cases lost

65 536
row limit, silently applied

~50 000
contacts never traced

What does a database give you that a file on someone's computer does not?

BBC News and *The Guardian*, 5 October 2020; Public Health England statement

Two seats, one airplane

Universität Konstanz

MOTIVATING EXAMPLE

Two travel agents, in two cities, book the last seat on the same flight in the **same second**. Each system checked availability, found one seat free, and sold it. Both passengers have a valid booking and a boarding pass.

- Neither program has a bug. Each is correct **on its own**.
- The failure only exists **between** them.

What has to be guaranteed for “check, then book” to be safe when thousands of people do it at once?

Whose number is right?

Universität Konstanz

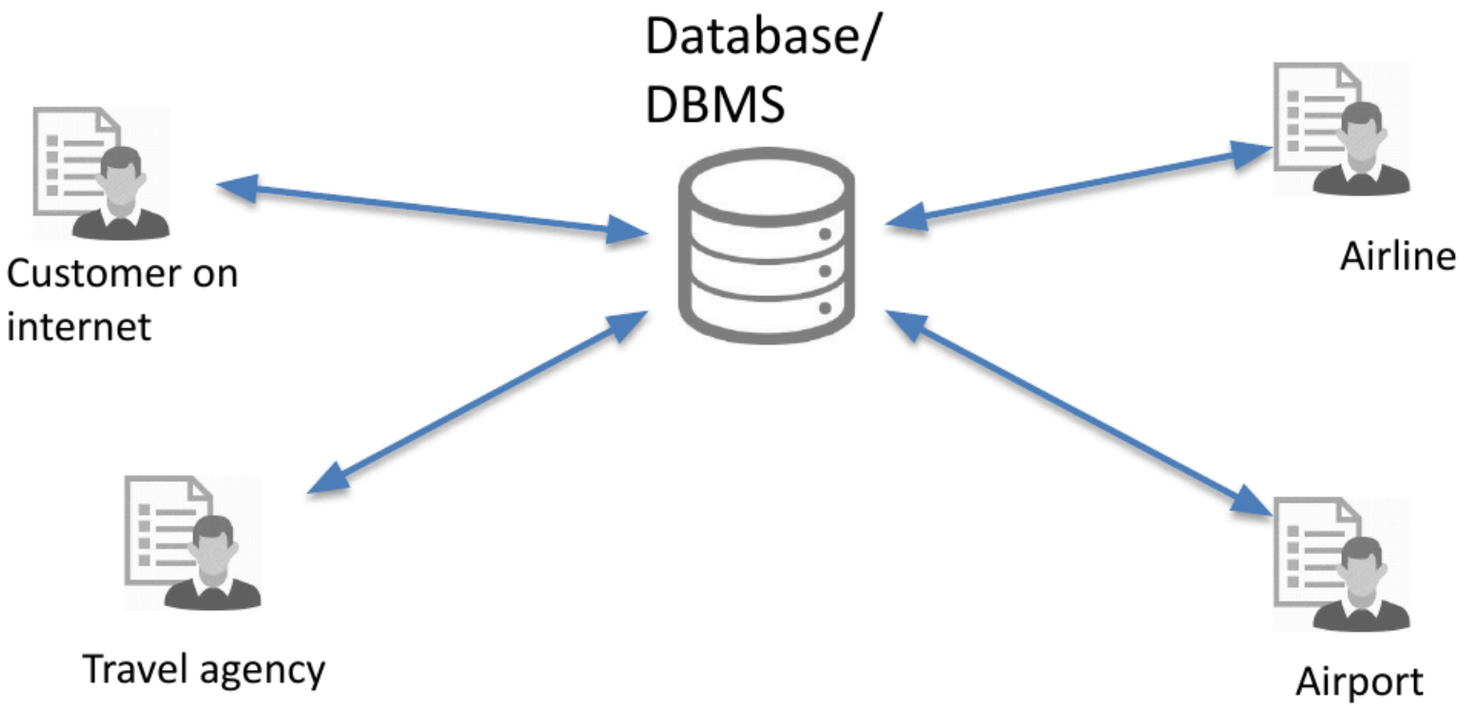
MOTIVATING EXAMPLE

Three people in a department each keep a copy of the participant list. One corrects a misspelled name, one adds the new wave, one removes a withdrawn respondent. Three months later the three files disagree about how many participants there are, by **47**.

- Everyone did their job correctly.
- There is **no version that is right**, and no way to reconstruct one.

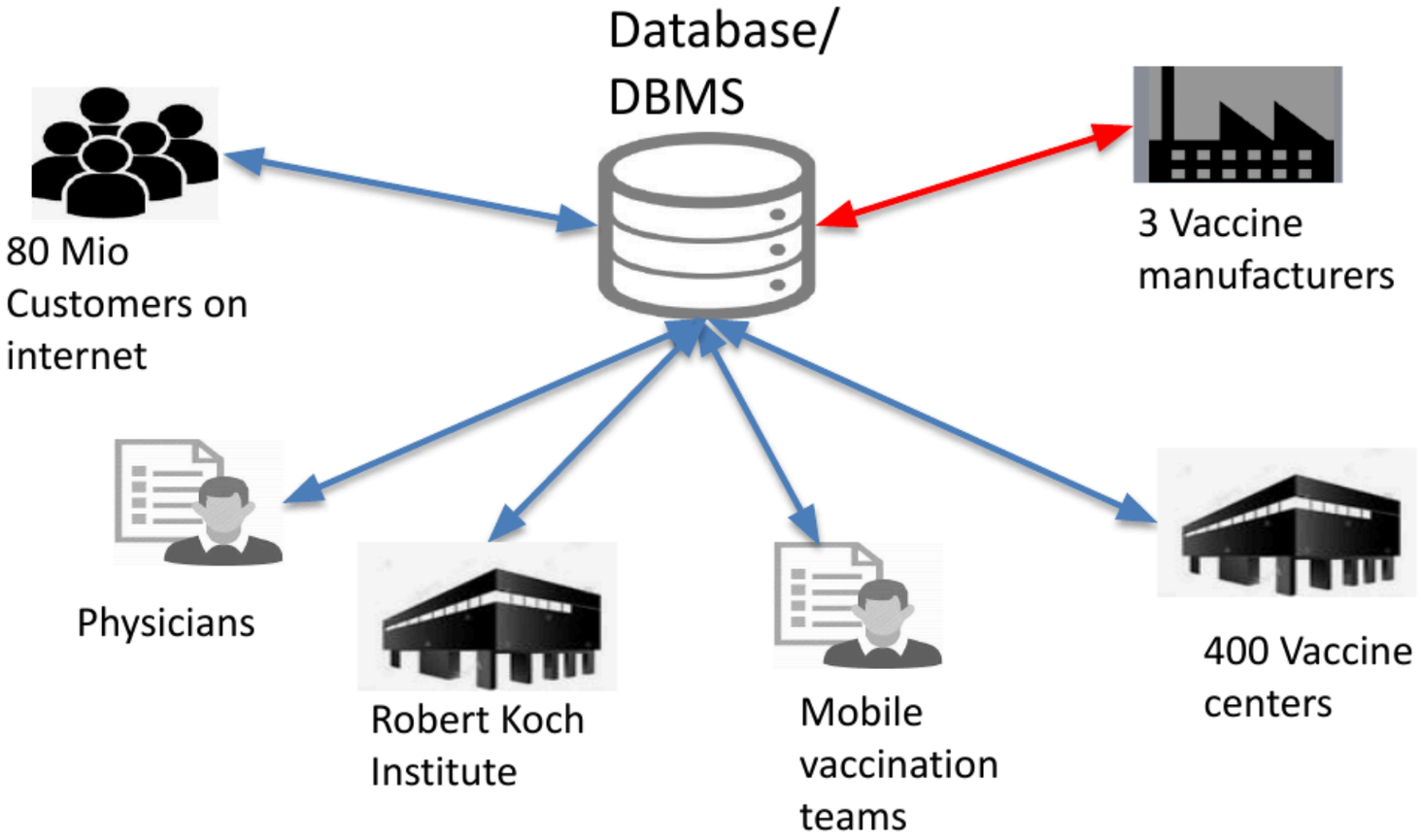
Where should a fact live, so that changing it once changes it everywhere?

Database example: Flight reservations



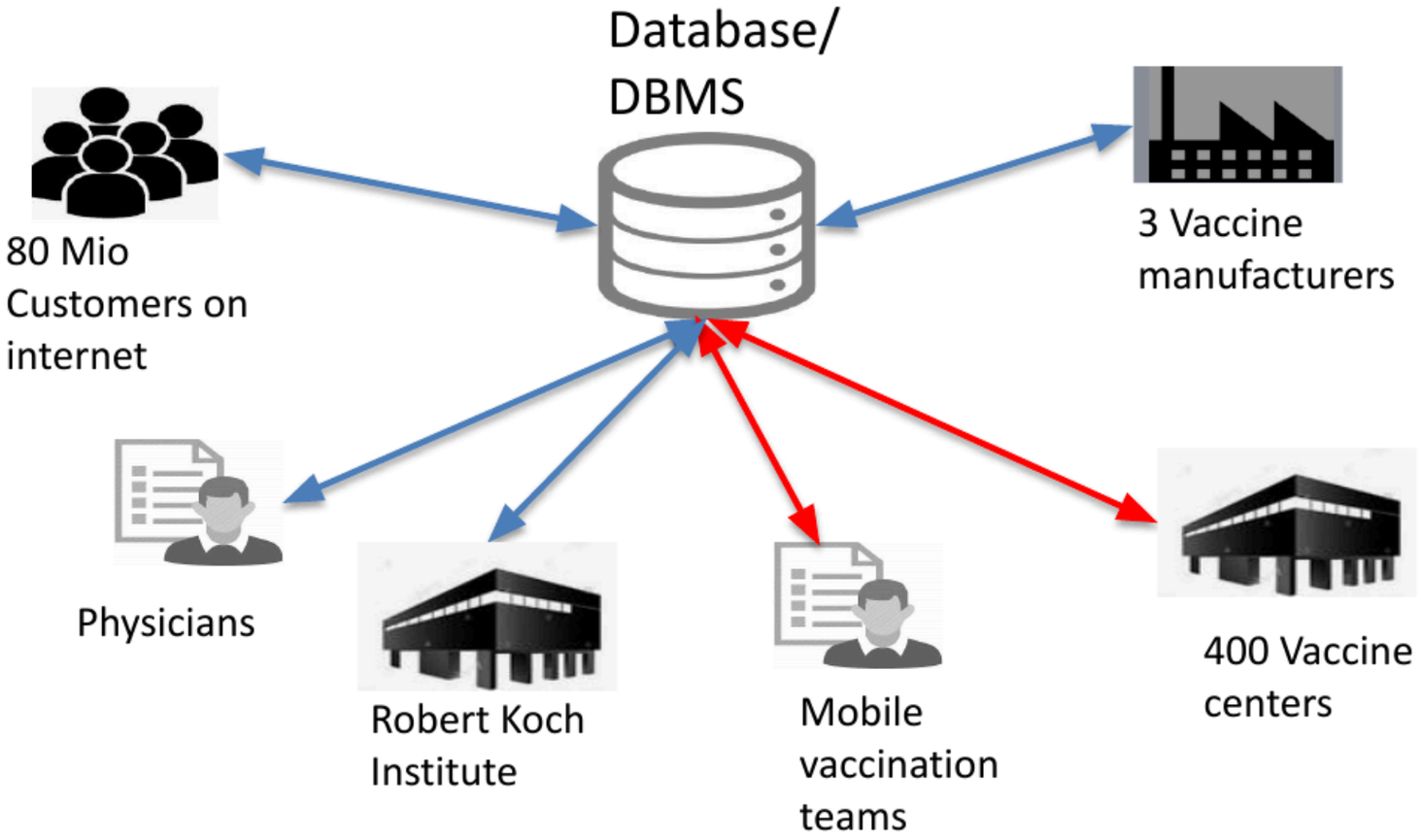
- Flight Reservation
- Seat reservation
- Special meals
- Cancel reservation
- Cancel flight
- Checkin
- Checkout
- Payment
- Security
- Reduced mobility

Database example: Vaccine distribution



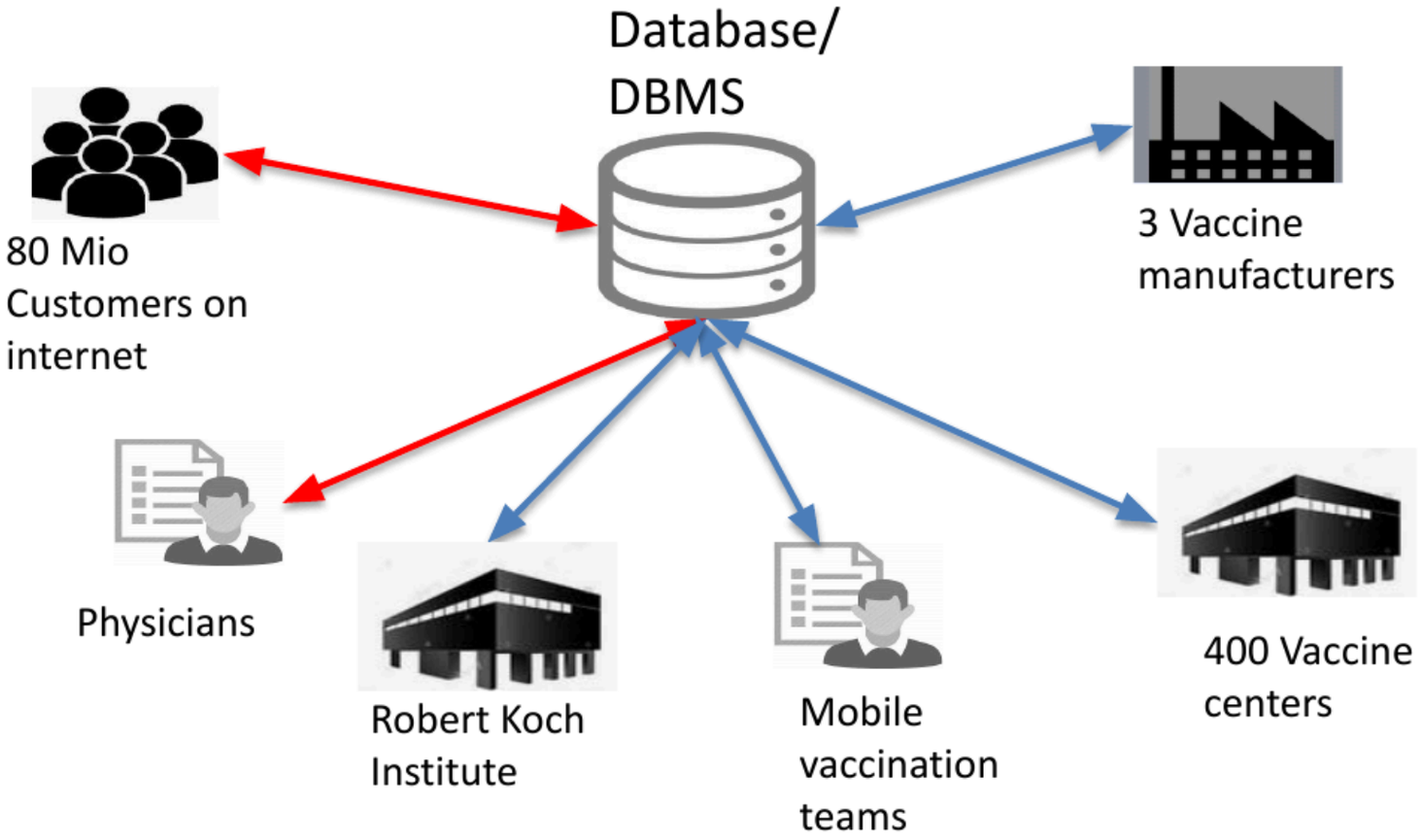
- Projection of vaccines
- Notify demands
- Divide vaccines
- Accept quota
- Schedules for vaccination
- Accept customer applications
- Report number of vaccinations
- Report remaining vaccines
- Create summary

Database example: Vaccine distribution



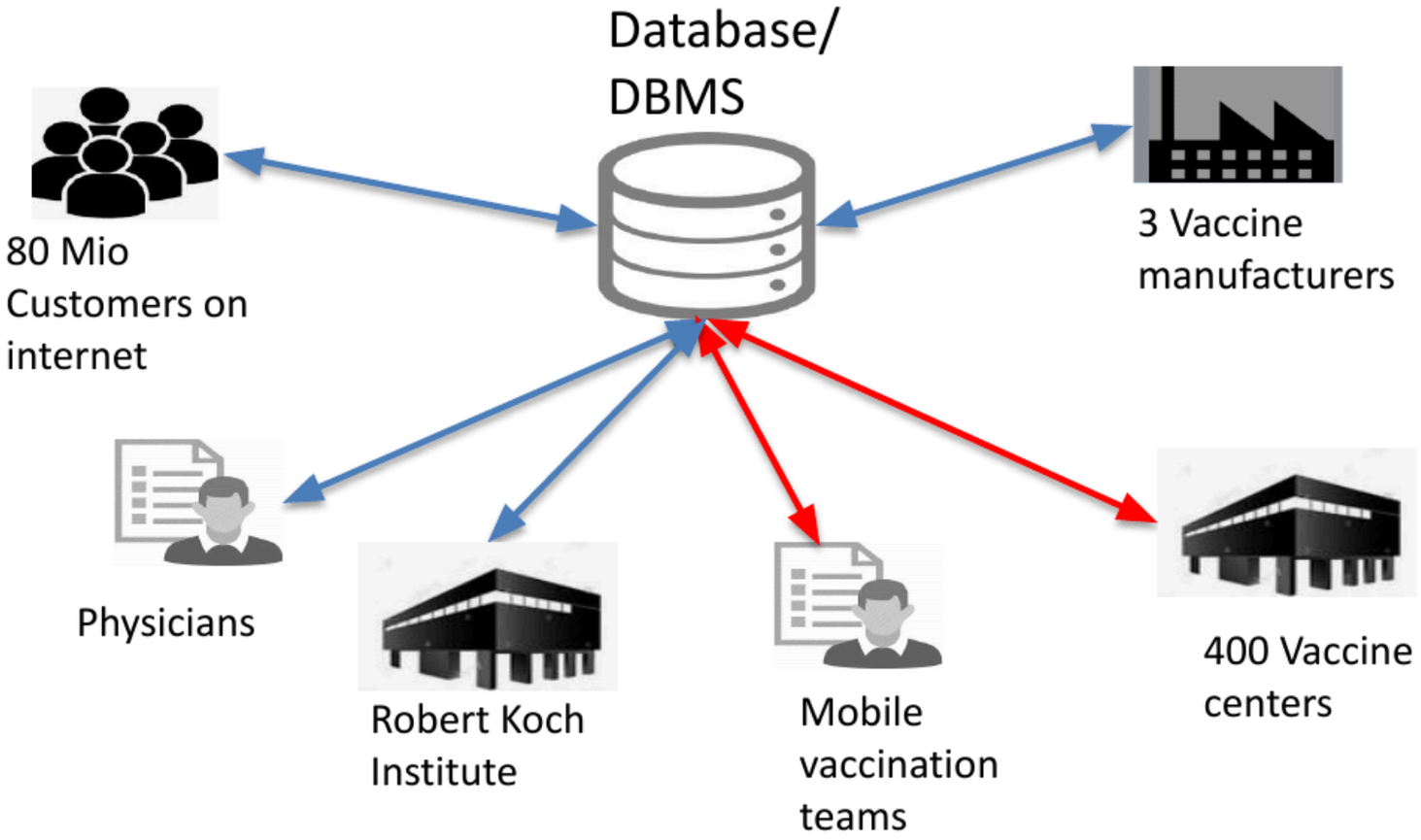
- Projection of vaccines
- Notify demands
- Divide vaccines
- Accept quota
- Schedules for vaccination
- Accept customer applications
- Report number of vaccinations
- Report remaining vaccines
- Create summary

Database example: Vaccine distribution



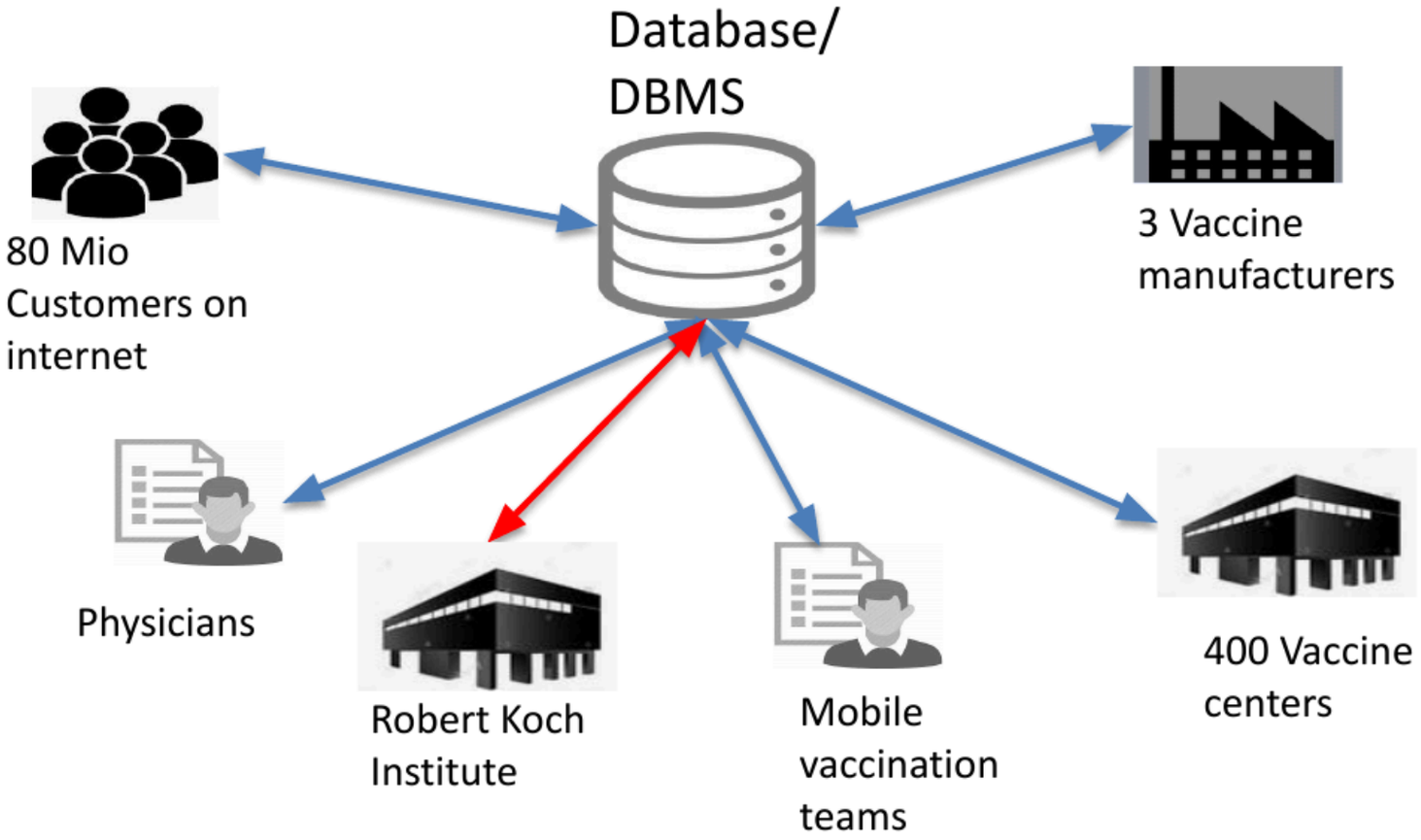
- Projection of vaccines
- Notify demands
- Divide vaccines
- Accept quota
- Schedules for vaccination
- Accept customer applications
- Report number of vaccinations
- Report remaining vaccines
- Create summary

Database example: Vaccine distribution



- Projection of vaccines
- Notify demands
- Divide vaccines
- Accept quota
- Schedules for vaccination
- Accept customer applications
- Report number of vaccinations
- Report remaining vaccines
- Create summary

Database example: Vaccine distribution



- Projection of vaccines
- Notify demands
- Divide vaccines
- Accept quota
- Schedules for vaccination
- Accept customer applications
- Report number of vaccinations
- Report remaining vaccines
- Create summary

Databases

Universität Konstanz

- **What is a database?**
  - Informal definition: an organized collection of data
    - an Excel table is a (very simple) database
    - enables fast access and the ability to access data in a structured way
- **What is a database management system (DBMS)?**
  - software application that enables interaction between the user and the actual database
  - implements defining, creating, querying, updating and administering a database
  - data may be distributed at different locations
  - examples: MySQL, PostgreSQL, Access, Oracle

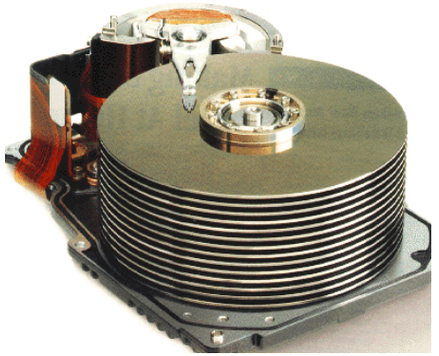
Databases and DBMS

Universität Konstanz

Definition: Database

A **database (DB)** is a set of related data and the formal organization and interdependence of these data. Access and manipulation of data are provided through a corresponding **database management system (DBMS)**.





A frequent icon to symbolize a database, represented by a hard drive disk platter

Characteristics of DBMS

Name	Birth	Class	Product	Num
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…
…	…	…	…	…

Characteristics of database management systems

- **data definition:** create, modify, delete the definitions defining the organization of data, i.e., the relationship among entries in the database

Characteristics of DBMS

Name	Birth	Class	Product	Num
Hans	1998	T1	Milk	00111
Fred	1999	T1	Tea	00222
Hans	1998	T1	Tea	00222
Anna	2000	W1	Milk	00111
Pat	1998	W1	Tea	00222
Hans	1998	T1	Milk	00111
Anna	2000	W1	Milk	00111
Fred	1999	T1	Milk	00111

Characteristics of database management systems

- **data definition:** create, modify, delete the definitions defining the organization of data, i.e., the relationship among entries in the database
- **updating:** insert, modify and delete the actual content, i.e., individual data entries in the database

Characteristics of DBMS

Name	Birth	Class	Product	Num
Hans	1998	T1	Milk	00111
Fred	1999	T1	Tea	00222
Hans	1998	T1	Tea	00222
Anna	2000	W1	Milk	00111
Pat	1998	W1	Tea	00222
Hans	1998	T1	Milk	00111
Anna	2000	W1	Milk	00111
Fred	1999	T1	Milk	00111

Product = Tea and Birth < 1999 ?

Characteristics of database management systems

- **data definition:** create, modify, delete the definitions defining the organization of data, i.e., the relationship among entries in the database
- **updating:** insert, modify and delete the actual content, i.e., individual data entries in the database
- **retrieval or querying:** return information, given specific query inputs, that can be used by any downstream application

Characteristics of DBMS

Name	Birth	Class	Product	Num
Hans	1998	T1	Milk	00111
Fred	1999	T1	Tea	00222
Hans	1998	T1	Tea	00222
Anna	2000	W1	Milk	00111
Pat	1998	W1	Tea	00222
Hans	1998	T1	Milk	00111
Anna	2000	W1	Milk	00111
Fred	1999	T1	Milk	00111

Characteristics of database management systems

- **data definition:** create, modify, delete the definitions defining the organization of data, i.e., the relationship among entries in the database
- **updating:** insert, modify and delete the actual content, i.e., individual data entries in the database
- **retrieval or querying:** return information, given specific query inputs, that can be used by any downstream application
- **administration:** user management, data security, performance, data integrity, concurrency, protection against database corruption and system failure

Why to use a database?

Universität Konstanz

- **data independence:** applications (and users) do not need to worry about how data are structured and stored
  - *logical data independence:* protection from changes to the logical structure of the data, e.g. add a new entity or attribute without affecting usage
  - *physical data independence:* protection from physical layout changes, e.g. the user does not have to care about information coding, disk space, etc.
- **standardization**
  - data formats, methods of accessing data, etc. are platform independent
    - recall: problems parsing a `.csv` file with conflict events in Python
  - flexible output schemas serving different users and purposes
- **concurrency**
  - allows simultaneous usage by many users without the problem of conflicting updates
  - robust protocols that avoid database corruption

Database Transactions: ACID properties

Universität Konstanz

A **transaction** is a series of data manipulations that must not be interrupted. All operations in a transaction have to be fully performed or completely rewound.

Database transactions have the following properties:

- **A**tomicity
  - each transaction stands on its own; a rewind is possible
- **C**onsistency
  - ensures that any transaction that succeeds maintains the validity of the database
- **I**solation
  - ensures that it does not matter whether we execute our transactions serially or concurrently
- **D**urability
  - requires that any transaction that has completed will survive permanently (also on error, crash, loss of power)

Relational Databases

- Databases
- **Relational Databases**
- Structured Query Language (SQL)

Database Models

Database models fundamentally define database structure

- set the logical structure of the database in terms of data storage, organization and manipulation
- ultimately about how the relationship between related information is represented
- examples:
  - **flat model**, a.k.a. spreadsheets:
    - a single, two-dimensional array of data elements
    - members of a row are related, i.e. an “entry”
    - members of a column describe the same property or characteristic
  - **relational model**:
    - representation through sets of linked tables that share a common key
    - intuition: many spreadsheets storing specific pieces of information, linked via a unique key

Flat model example

Name	Birth	Class	Product	Productnr	Date
Hans	1998	T1	Milk	00111	06.03.2020
Fred	1999	T1	Tea	00222	06.03.2020
Hans	1998	T1	Tea	00222	06.03.2020
Anna	2000	W1	Milk	00111	06.03.2020
Verena	1998	W1	Tea	00222	07.03.2020
Hans	1998	T1	Milk	00111	07.03.2020
Anna	2000	W1	Milk	00111	08.03.2020
Fred	1999	T1	Milk	00111	08.03.2020

Flat model example

Name	Birth	Class	Product	Productnr	Date
Hans	1998	T1	Milk	00111	06.03.2020
Fred	1999	T1	Tea	00222	06.03.2020
Hans	1998	T1	Tea	00222	06.03.2020
Anna	2000	W1	Milk	00111	06.03.2020
Verena	1998	W1	Tea	00222	07.03.2020
Hans	1998	T1	Milk	00111	07.03.2020
Anna	2000	W1	Milk	00111	08.03.2020
Fred	1999	T1	Milk	00111	08.03.2020

Data is repeated across the table

- Inefficient use of space
- Problems updating: what if the Productnr of Milk is now 00333?

Building a relational database

Universität Konstanz

Table purchase

Nr	Date	Name	Product
1	06.03.2020	1	1
2	06.03.2020	2	2
3	06.03.2020	1	2
4	06.03.2020	3	1
5	07.03.2020	4	2
6	07.03.2020	1	1
7	08.03.2020	3	1
8	08.03.2020	2	1

Primary key

Table Names

Nr	Name	Birth	Class
1	Hans	1998	T1
2	Fred	1999	T1
3	Anna	2000	W1
4	Verena	1998	W1

Table Product

Nr	Product	Productnr
1	Milk	00111
2	Tea	00222

Building a relational database

Universität Konstanz

Table purchase

Nr	Date	Name	Product
1	06.03.2020	1	1
2	06.03.2020	2	2
3	06.03.2020	1	2
4	06.03.2020	3	1
5	07.03.2020	4	2
6	07.03.2020	1	1
7	08.03.2020	3	1
8	08.03.2020	2	1

Foreign keys reference primary keys (on another table or the same table)

Table Names

Nr	Name	Birth	Class
1	Hans	1998	T1
2	Fred	1999	T1
3	Anna	2000	W1
4	Verena	1998	W1

Table Product

Nr	Product	Productnr
1	Milk	00111
2	Tea	00222

Relational database: normal form example

Universität Konstanz

Table purchase

Nr	Date	Name	Product
1	06.03.2020	1	1
2	06.03.2020	2	2
3	06.03.2020	1	2
4	06.03.2020	3	1
5	07.03.2020	4	2
6	07.03.2020	1	1
7	08.03.2020	3	1
8	08.03.2020	2	1

Table Names

Nr	Name	Birth	Class
1	Hans	1998	1
2	Fred	1999	1
3	Anna	2000	2
4	Verena	1998	2

Table Classes

Nr	Name
1	T1
2	W1

Table Product

Nr	Product	Productnr
1	Milk	00111
2	Tea	00222

- Relational database designs can be standardized using **normal forms**
- For example here: Class should be its own table

Relational database: normal form example

Universität Konstanz

Table purchase

Nr	Date	Name	Product
1	06.03.2020	1	1
2	06.03.2020	2	2
3	06.03.2020	1	2
4	06.03.2020	3	1
5	07.03.2020	4	2
6	07.03.2020	1	1
7	08.03.2020	3	1
8	08.03.2020	2	1

Table Names

Nr	Name	Birth	Class
1	Hans	1998	1
2	Fred	1999	1
3	Anna	2000	2
4	Verena	1998	2

Table Classes

Nr	Name
1	T1
2	W1

Table Product

Nr	Product	Productnr
1	Milk	00111
2	Tea	00222

Now we can update the name of a class to another one without having to edit the purchase table

Structured Query Language (SQL)

- Databases
- Relational Databases
- **Structured Query Language (SQL)**

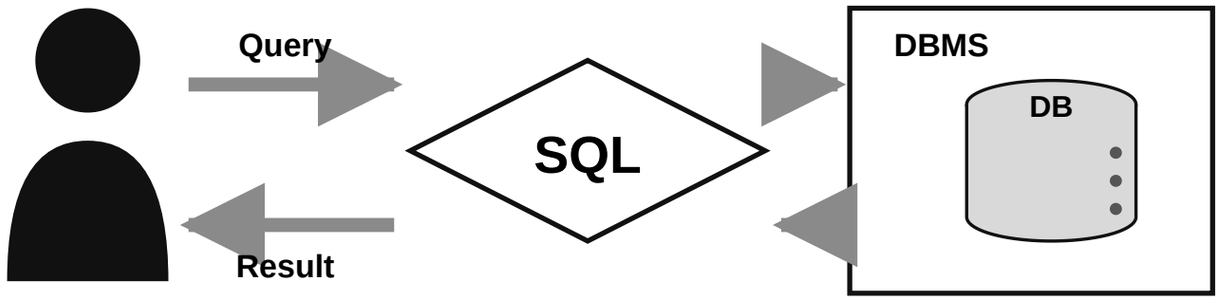
SQL Programming Language

Universität Konstanz

- special-purpose language specifically developed for managing data through relational database systems
- consists of three (co-dependent) language definitions
  - **data definition language (DDL)**
    - define relational schemata
    - create / alter / delete tables and their attributes
  - **data manipulation language (DML)**
    - insert / delete / modify entries in tables
    - query one or more tables
  - **data control language (DCL)**
    - control access to data stored in the database
    - grant / revoke / modify user access rights

SQL Programming Language

- SQL is a very **high-level** programming language
  - performance is good because the actual operations are optimized well
  - many different standards, but typical DBMS support all
  - execution is highly parallelizable by construction
- SQL is a **declarative** language
  - You define *what* you want, the DBMS takes care of *how* to do it
  - You can also do “procedural SQL” with special instructions, but it is very inefficient



Tables in SQL

Universität Konstanz

Tables

A multiset of tuples coding the attributes specified by the database schema.

- Tables are also called **relations**
- Tuples are also called **rows**
- Attributes are also called **columns**

Product

PName	Price	Manufacturer
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

Tables in SQL

Universität Konstanz

Tables

A multiset of tuples coding the attributes specified by the database schema.

- Tables are also called **relations**
- Tuples are also called **rows**
- Attributes are also called **columns**

Product

PName	Price	Manufacturer
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

Table (relation)

Tables in SQL

Universität Konstanz

Tables

A multiset of tuples coding the attributes specified by the database schema.

- Tables are also called **relations**
- Tuples are also called **rows**
- Attributes are also called **columns**

Product

PName	Price	Manufacturer
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

Row (tuple)

Tables in SQL

Universität Konstanz

Tables

A multiset of tuples coding the attributes specified by the database schema.

- Tables are also called **relations**
- Tuples are also called **rows**
- Attributes are also called **columns**

Can we have the same row twice?

Product

PName	Price	Manufacturer
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

Row (tuple)

Tables in SQL

Universität Konstanz

Tables

A multiset of tuples coding the attributes specified by the database schema.

- Tables are also called **relations**
- Tuples are also called **rows**
- Attributes are also called **columns**

Product

PName	Price	Manufacturer
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

Column (attribute)

Tables in SQL

Universität Konstanz

Tables

A multiset of tuples coding the attributes specified by the database schema.

- Tables are also called **relations**
- Tuples are also called **rows**
- Attributes are also called **columns**

Product

PName	Price	Manufacturer
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

Schema

SQL Data types

Universität Konstanz

- **Data Types**
  - atomic types:
    - characters: `CHAR(20)`, `VARCHAR(50)`
    - numbers: `INT`, `BIGINT`, `SMALLINT`, `FLOAT`
    - others: `MONEY`, `DATETIME`, …
  - every attribute (column) must have an **atomic** type
    - consequently tables in SQL are always **flat**: cells (in principle) should not contain lists or matrices

SQL Schema

Universität Konstanz

- **Table Schema**
  - the schema of a table is its name, attributes and their types
    - in our example:

      `Product: (PName: string, Price: float, Manufacturer: string)`
  - a **key** is a minimal subset of attributes that acts as a unique identifier for tuples in a relation
    - an implicit constraint on which tuples can be in a relation
      - if two tuples in the same table agree on the values of the key, they must be the same tuple!
    - notation: in the schema the key is underlined – `PName` in our example

SQL data definition language

Universität Konstanz

- **CREATE:**

```
1 CREATE TABLE extable(col1 INTEGER, col2 VARCHAR(50), PRIMARY KEY (col1));
```

extable

col1	col2

Initializes an empty table with the specified schema: name, attributes and keys

SQL data manipulation language

Universität Konstanz

- **INSERT**

```
1 INSERT INTO phone_book (name, number) VALUES ('John Doe', '555-1212');
```

phone_book

name	number
Jane Doe	333-333
Max Mustermann	4949-4900

before

phone_book

name	number
Jane Doe	333-333
Max Mustermann	4949-4900
John Doe	**555-1212**

after

SQL data manipulation language

Universität Konstanz

- **UPDATE**

```sql
1 UPDATE Table1 SET C1 = C1 + 1 WHERE C2 = 'a';
```

Table1

C1	C2
1	a
2	b
3	a
4	x

before

Table1

C1	C2
2	a
2	b
4	a
4	x

after

SQL data manipulation language

Universität Konstanz

- **DELETE**

```
1 DELETE FROM example WHERE column2 = 'N';
```

- **DROP**

```
1 DROP TABLE table_name;
```

- Many things can be dropped: indices, columns, whole databases…
- **ROLLBACK**
  - Discards all data changes since the last `COMMIT` or `ROLLBACK`
  - By default, a set of manipulations are part of a *transaction* that is executed and finished when we call `COMMIT`

SQL tutorial: https://www.w3schools.com/sql/default.asp

SQL Queries

Universität Konstanz

SQL **queries** allow us to define what data to receive from a database.

Basic form of a SQL query:

```
1 SELECT <attributes> FROM <one or more relations> WHERE <conditions>
```

SQL Queries: selection

Universität Konstanz

selection example:

```
1 SELECT * FROM Product WHERE Manufacturer = 'GizmoWorks'
```

Product

PName	Price	Manufacturer
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

SQL Queries: selection + projection

Universität Konstanz

selection + projection example:

```
1 SELECT PName, Price FROM Product WHERE Manufacturer = 'GizmoWorks'
```

Product

PName	Price	Manufacturer
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

Aggregation in SQL

Universität Konstanz

```
1 SELECT AVG(Price) FROM Product WHERE Manufacturer = 'GizmoWorks'
```

Product

PName	Price	Manufacturer
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

AVG(Price)
$24.99

Grouping and aggregating in SQL


```
1 SELECT Manufacturer, AVG(Price) FROM Product GROUP BY Manufacturer
```

Product

PName	Price	Manufacturer
Gizmo	$19.99	GizmoWorks
Powergizmo	$29.99	GizmoWorks
SingleTouch	$149.99	Canon
MultiTouch	$203.99	Hitachi

Manufacturer	AVG(Price)
GizmoWorks	$24.99
Canon	$149.99
Hitachi	$203.99

More about aggregation and grouping in the tutorial sessions

SQL manual join

Universität Konstanz

- A **join** resembles a merge or product operation in other languages

```
1 SELECT * FROM purchase, Product WHERE purchase.Product = Product.Nr;
```

purchase

Nr	Date	Name	Product
1	06.03.2020	1	1
2	06.03.2020	2	2
3	06.03.2020	1	2
4	06.03.2020	3	1
5	07.03.2020	4	3
6	07.03.2020	1	3
7	08.03.2020	3	4
8	08.03.2020	2	4

Product

Nr	Product	Productnr
1	Milk	00111
2	Tea	00222

SQL manual join

```
1 SELECT * FROM purchase, Product WHERE purchase.Product = Product.Nr;
```

Result:

Nr	Date	Name	purchase.Product	Product.Product	Productnr
1	06.03.2020	1	1	Milk	00111
2	06.03.2020	2	2	Tea	00222
3	06.03.2020	1	2	Tea	00222
4	06.03.2020	3	1	Milk	00111

- Why are there now only four rows when purchase had 8?
- Why are there new column names?
- How many rows would we get with `SELECT * FROM purchase, Product;`?

Better naming of columns

Universität Konstanz

purchase

ID	Date	Name	ProductID
1	06.03.2020	1	1
2	06.03.2020	2	2
3	06.03.2020	1	2
4	06.03.2020	3	1
5	07.03.2020	4	3
6	07.03.2020	1	3
7	08.03.2020	3	4
8	08.03.2020	2	4

Product

ProductID	Product	Productnr
1	Milk	00111
2	Tea	00222

- References to keys of other columns are called **foreign keys**
- Naming can be done in a smart way to simplify expected joins

SQL INNER JOIN

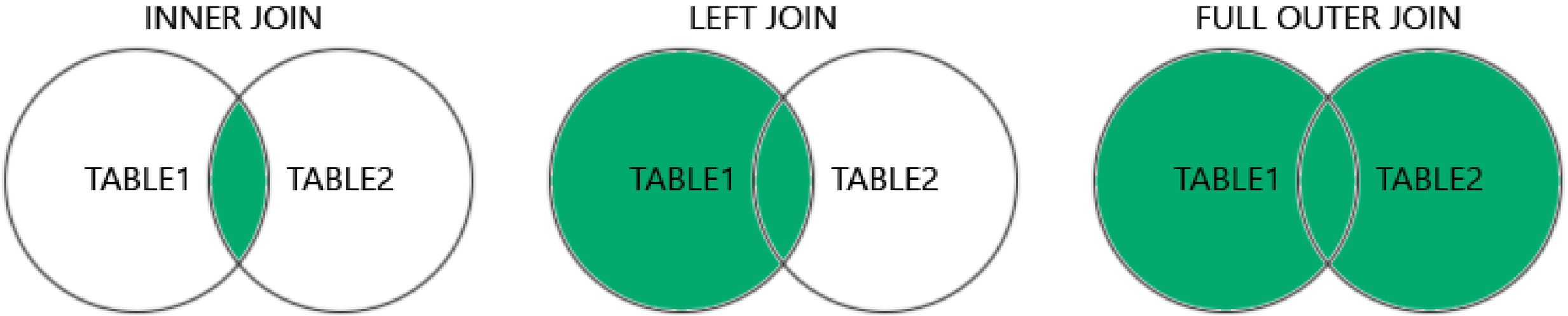
Universität Konstanz

```
1 SELECT * FROM purchase INNER JOIN Product USING (ProductID);
```

Result:

ID	Date	Name	ProductID	Product	Productnr
1	06.03.2020	1	1	Milk	00111
2	06.03.2020	2	2	Tea	00222
3	06.03.2020	1	2	Tea	00222
4	06.03.2020	3	1	Milk	00111

SQL other JOIN operations

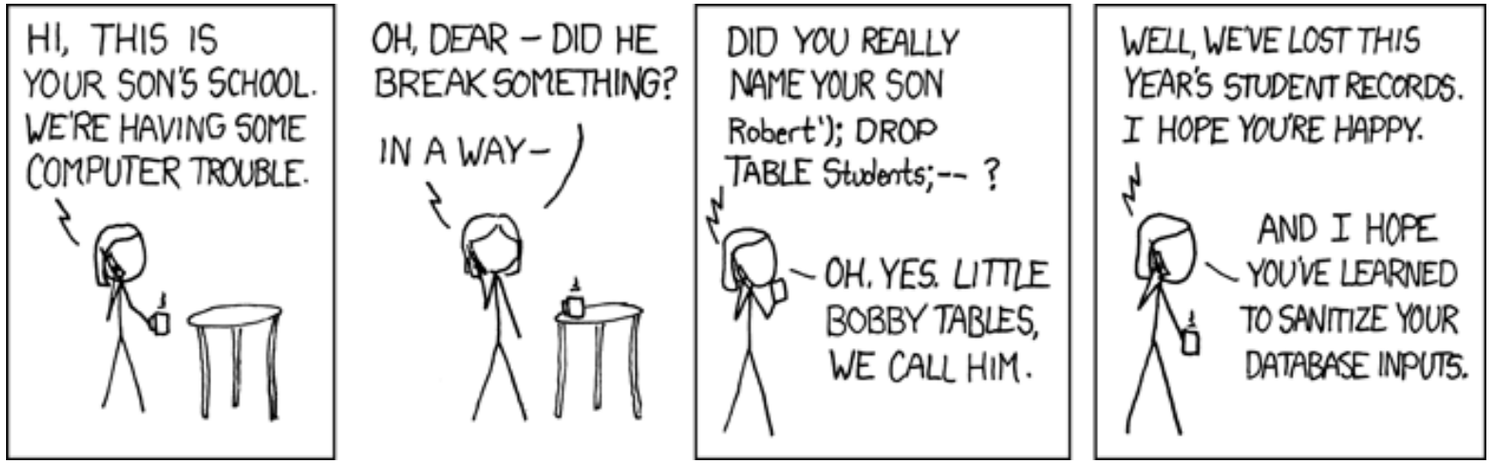


- There are different kinds of joins based on **set operations**
- If values are missing from one of the columns, they are set to `NULL`

More on the SQL tutorial: https://www.w3schools.com/sql/default.asp

SQL injections





- A **SQL injection** is a security attack on a database that inputs SQL code as user input (e.g. a user name)
- If the database permissions are not well set, it could be executed
- It can be used to delete the database, for illicit data export, or to gain access to other parts of the system

XKCD 327, *Exploits of a Mom*: https://xkcd.com/327/

Summary

Universität Konstanz

- **Databases**
  - Technology to centralize, manage and provide access to data
  - A DBMS is software that handles the DB while we just operate with it
  - Transactions are atomic, consistent, isolated and durable (ACID)
- **Relational Databases**
  - Tables linked through primary and foreign keys: each fact is stored once, so updates stay consistent
- **Structured Query Language (SQL)**
  - The most used language for relational databases, and a declarative one
  - Define tables, change their rows, and query them with selection, projection, aggregation and joins

Next session: Parallel Programming